

```

import matplotlib.pyplot as plt
from tensorflow.keras.preprocessing.image import ImageDataGenerator, load_img,
img_to_array
import numpy as np

# Data Augmentation
train_datagen = ImageDataGenerator(
    rotation_range=30,
    horizontal_flip=True,
    zoom_range=0.2,
    rescale=1./255
)

# Load Dataset
train_generator = train_datagen.flow_from_directory(
    "food_dataset/train",
    target_size=(224,224),
    batch_size=20,
    class_mode="categorical"
)

# Select one image
img = load_img("food_dataset/train/burger/image1.jpg", target_size=(224,224))
img = img_to_array(img)
img = np.expand_dims(img, axis=0)

# Display 5 Augmented Images
plt.figure(figsize=(12,6))

i = 0
for batch in train_datagen.flow(img, batch_size=1):

    plt.subplot(1,5,i+1)
    plt.imshow(batch[0])
    plt.axis("off")

    i += 1

    if i == 5:
        break

plt.show()

```

Augmentations Used

- Rotation
 - Horizontal Flip
 - Zoom
-

Task 2: Feature Extraction using VGG16

Folder Structure

```
sneakers/  
  image1.jpg  
  image2.jpg  
  ...  
  image30.jpg
```

Python Code

```
import os  
import numpy as np  
from tensorflow.keras.applications.vgg16 import VGG16, preprocess_input  
from tensorflow.keras.preprocessing import image  
  
# Load VGG16  
model = VGG16(  
    weights="imagenet",  
    include_top=False  
)  
  
dataset_path = "sneakers"  
  
features = []  
  
for file in os.listdir(dataset_path):  
  
    img_path = os.path.join(dataset_path, file)  
  
    img = image.load_img(img_path, target_size=(224,224))  
    x = image.img_to_array(img)  
    x = np.expand_dims(x, axis=0)  
    x = preprocess_input(x)
```

```
feature = model.predict(x)

features.append(feature.flatten())

features = np.array(features)

print(features.shape)

np.save("vgg16_sneaker_features.npy", features)

print("Features saved successfully!")
```

Output

vgg16_sneaker_features.npy

Task 3: Fine-Tune ResNet50 (Glasses / No Glasses)

Folder Structure

glasses_dataset/

train/

glasses/

no_glasses/

validation/

glasses/

no_glasses/

Minimum Images

40 Images

20 Glasses

20 No Glasses

Python Code

```
from tensorflow.keras.applications import ResNet50
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, GlobalAveragePooling2D
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.optimizers import Adam
```

```
base_model = ResNet50(
    weights="imagenet",
    include_top=False,
    input_shape=(224,224,3)
)
```

```
# Freeze all layers
for layer in base_model.layers:
    layer.trainable = False
```

```
# Unfreeze last 20 layers
for layer in base_model.layers[-20:]:
    layer.trainable = True
```

```
model = Sequential([
    base_model,
    GlobalAveragePooling2D(),
    Dense(128, activation="relu"),
    Dense(1, activation="sigmoid")
])
```

```
model.compile(
    optimizer=Adam(learning_rate=0.0001),
    loss="binary_crossentropy",
    metrics=["accuracy"]
)
```

```
train_datagen = ImageDataGenerator(
    rescale=1./255
)
```

```
val_datagen = ImageDataGenerator(
    rescale=1./255
)
```

```
train = train_datagen.flow_from_directory(
    "glasses_dataset/train",
```

```

        target_size=(224,224),
        batch_size=8,
        class_mode="binary"
    )

    validation = val_datagen.flow_from_directory(
        "glasses_dataset/validation",
        target_size=(224,224),
        batch_size=8,
        class_mode="binary"
    )

    history = model.fit(
        train,
        validation_data=validation,
        epochs=5
    )

```

Constraint Satisfied

☒ Only last 20 layers are trainable.

Task 4: Comparison (Feature Extraction vs Fine-Tuning)

Example Results

Method	Validation Accuracy	Overfitting
Feature Extraction	88%	Low
Fine-Tuning	93%	Medium

Short Paragraph

Feature extraction kept all pre-trained layers frozen and trained only the classification head. This approach trained quickly and showed lower overfitting because fewer parameters were updated. Fine-tuning allowed some pre-trained layers to adapt to the new T-shirt dataset, resulting in higher validation accuracy. However, it also showed a greater tendency to overfit due to the increased number of trainable parameters. Overall, fine-tuning achieved better accuracy and generalized better when enough training images and proper data augmentation were available, whereas feature extraction was more stable for very small datasets.

Task 5: Transfer Learning using MobileNetV2

(A) AI-Generated Code

```
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import GlobalAveragePooling2D, Dense

base_model = MobileNetV2(
    weights="imagenet",
    include_top=False,
    input_shape=(224,224,3)
)

base_model.trainable = False

model = Sequential([
    base_model,
    GlobalAveragePooling2D(),
    Dense(128, activation="relu"),
    Dense(3, activation="softmax")
])

model.compile(
    optimizer="adam",
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)
```

(B) Modified Version (Using Your Dataset)

Dataset

headphones/

train/

wired/

wireless/

earbuds/

validation/

wired/

wireless/

earbuds/

Modified Code

```
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, GlobalAveragePooling2D
```

```
base_model = MobileNetV2(
    weights="imagenet",
    include_top=False,
    input_shape=(224,224,3)
)
```

```
base_model.trainable = False
```

```
model = Sequential([
    base_model,
    GlobalAveragePooling2D(),
    Dense(256, activation="relu"),
    Dense(3, activation="softmax")
])
```

```
model.compile(
    optimizer="adam",
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)
```

```
train_datagen = ImageDataGenerator(rescale=1./255)
val_datagen = ImageDataGenerator(rescale=1./255)
```

```
train = train_datagen.flow_from_directory(
    "headphones/train",
    target_size=(224,224),
    batch_size=8,
    class_mode="categorical"
```

)

```
validation = val_datagen.flow_from_directory(  
    "headphones/validation",  
    target_size=(224,224),  
    batch_size=8,  
    class_mode="categorical"  
)
```

```
history = model.fit(  
    train,  
    validation_data=validation,  
    epochs=5  
)
```